Web3 支付：Escrow 托管 + 钱包签名 + 链上监听

gomall 把” 第三方支付” 换成” 可审计代码” 的工程链路

gomall · 业务侧 deck

今日大纲

业务目的：传统支付吃不下的那批订单

Escrow 合约：把” 第三方” 换成代码

钱包签名：EIP-191 personal_sign + nonce 防重放

链上对账：链是异步的

Gas 与费用模型：每笔订单实际花多少链费

Layer 2：把链费降下来，把 TPS 抬上去

多签仲裁：从单 EOA 升到 2-of-3 多签

稳定币 ERC-20 直付：approve + transferFrom

业务边界：业务码 / 角色 / 合规

业务目的：传统支付吃不下的那批订单

为什么 gomall 引入 Web3 支付

- **加密原生用户**：DeFi 用户钱包里全是链上资产，刷卡通道用不上。
- **跨境结算**：传统通道 1–3% 手续费 + T+1 到账，跨境争议解决依赖银行 SWIFT。
- **资金托管增信**：买卖双方互不信任的大额订单，把”中介”从银行换成审计过的合约。
- **业务收益**：开新渠道，不是替代旧通道，跑在传统 `/paydown` 旁边。

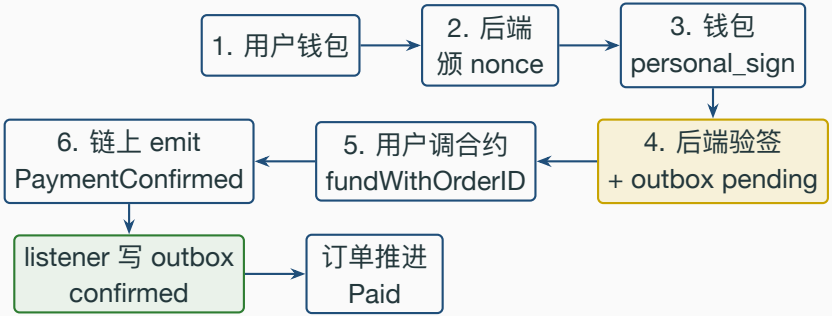
routes/router.go:113–127 paydown（fiat）与 paydown/crypto（web3）平行挂载

传统第三方支付 vs Escrow 托管

维度	第三方支付	Web3 escrow
信任锚	银行 / 支付牌照	智能合约 + 链
结算时间	T+0 / T+1	1 块 30–60s, 6 块 ≈ 90s
跨境费用	1–3%	gas 固定 + 兑换损耗
退款主导	平台仲裁	合约 + 仲裁人
身份证明	实名 + 卡密	钱包私钥 EIP-191 签名

Web3 不替代第三方支付，而是**补齐**传统通道吃不下的订单。*contracts/README.md:1–36* 合约职责与状态机不变式

端到端 6 步链路

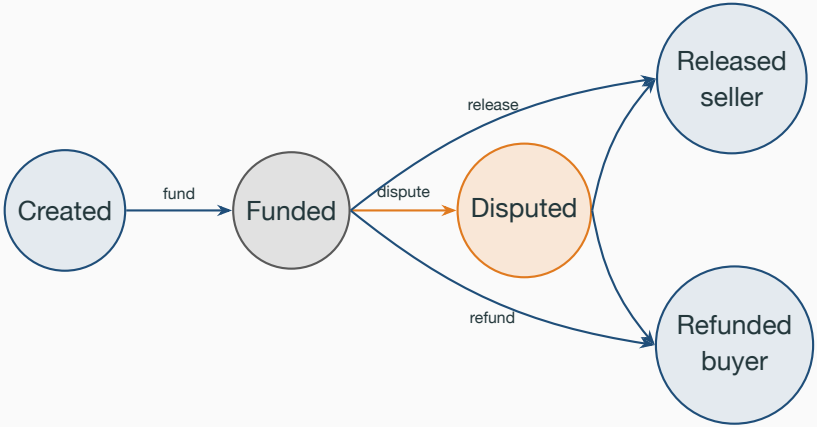


钱进合约前订单永远 pending；钱进合约后由 listener 兜底推进，避免”先付款后失踪”。*routes/router.go:123–127*

GET paydown/crypto/nonce + POST paydown/crypto

Escrow 合约：把” 第三方” 换成代码

Escrow 状态机：5 个状态、3 个角色



不变式：**Created** → **Funded** 只能发生一次（`inState(Created)` 守卫）；**Funded / Disputed** → **Released | Refunded** 终态不可逆；所有 immutable 参数（buyer/seller/arbiter/amount）部署后无法改。*contracts/Escrow.sol:13–19 enum；:68–82 fund/fundWithOrderID；:85–107 release/refund/dispute*

合约状态 ↔ 订单状态映射

合约状态	订单状态	触发动作
Created	已下单未支付	合约部署完成
Funded	已支付待发货	买家 `fundWithOrderID`
(链下)	商家发货 / 物流	链下流程，不影响合约
Released	已完成	买家 / 仲裁人 `release`
Refunded	已退款	卖家 / 仲裁人 `refund`
Disputed	售后争议	买家或卖家 `dispute`

链上只记” 钱在谁那里”；订单

详情 / 物流 / 对账都在链下。

合约的不变式 = 业务对外的最低承诺。*contracts/README.md:31–36* 合约不变式条款

Escrow.sol 关键路径：fundWithOrderID

Listing 1: *contracts/Escrow.sol:74–82 fundWithOrderID*

```
74 function fundWithOrderID(bytes32 _orderID)
75    external payable onlyBuyer inState(State.Created)
76 {
77    if (msg.value != amount) revert WrongAmount(amount, msg.value);
78    orderID = _orderID;
79    state = State.Funded;
80    emit Funded(msg.value);
81    emit PaymentConfirmed(_orderID, msg.sender, msg.value);
82 }
```

逐行解读：

- `onlyBuyer + inState(Created)`：只能 buyer 调，且只能在 Created 态调，状态机入口唯一。
- `msg.value != amount`：链上强校验金额，少 1 wei 也 revert，业务侧零信任。
- `orderID = _orderID`：链下订单号写进 storage，listener 拿它 join 业务库。

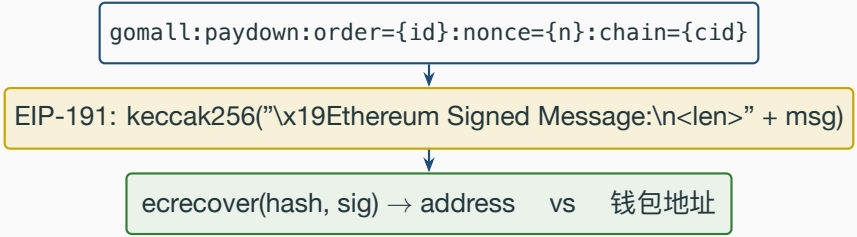
钱包签名：EIP-191 personal_sign + nonce 防重放

为什么不能把私钥交给后端

- 私钥 = 钱包的全部，给了后端就不再是”用户的钱包”。
- 业界做法：后端给一段消息 → 钱包客户端内签 → 后端 `ecrecover` 反推地址。
- 业务收益：
  - **不可否认**：地址匹配 = 用户授权过；
  - **零私钥泄漏面**：签名发生在钱包客户端内部；
  - **防钓鱼**：消息明文人类可读，钱包弹窗能让用户看清自己在签什么。

pkg/web3/signature/verify.go:1–13 包注释设计意图

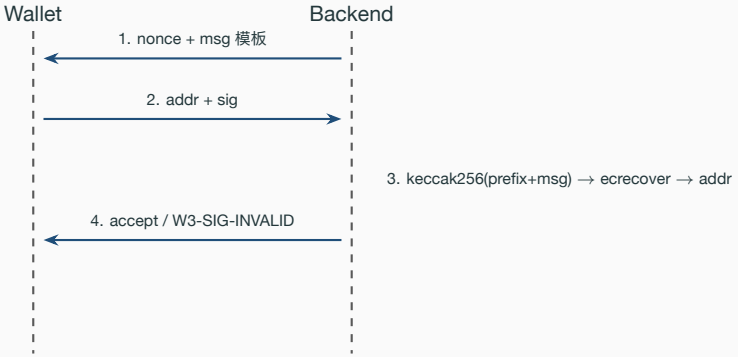
签名消息模板：chainID + orderID + nonce 全钉进去



- **chainID 入消息**：防一条签名在 mainnet / L2 之间互通造成跨链重放。
- **orderID + nonce**：绑定订单 + 防同链重放。
- **前缀 `gomall:paydown:`**：让用户在钱包弹窗里识别" 我在给 gomall 付款"。
- **v 值归一**：钱包 27/28，go-ethereum 还原 pubkey 要 0/1，必须归一。

service/payment_crypto.go:39–46 signMessageTemplate pkg/web3/signature/verify.go:31–67 VerifyPersonalSign

验签时序：4 拍完成” 是不是这个钱包”



全过程后端不接触私钥；

签名失败 → `W3-SIG-INVALID`；nonce 失效 → `W3-NONCE-EXPIRED`。

service/payment_crypto.go:104–124 VerifyAndPark 第 1–2 段

VerifyPersonalSign：EIP-191 + ecrecover 12 行

Listing 2: *pkg/web3/signature/verify.go:31–67*

```go
func VerifyPersonalSign(addr string, msg []byte, sig []byte) (bool, error) {
    if len(sig) != personalSigLen { return false, fmt.Errorf("...") }
    if !common.IsHexAddress(addr) { return false, errors.New("...") }
    rsv := make([]byte, personalSigLen)
    copy(rsv, sig)
    switch rsv[64] {
    case 27, 28: rsv[64] -= 27
    case 0, 1: // already normalized
    default: return false, fmt.Errorf("非法 v 值: %d", rsv[64])
    }
    hash := personalSignHash(msg) // EIP-191 prefix + keccak256
    pubKey, err := crypto.SigToPub(hash, rsv) // ecrecover
    if err != nil { return false, err }
    recovered := crypto.PubkeyToAddress(*pubKey)
    expected := common.HexToAddress(addr)
    return bytes.Equal(recovered.Bytes(), expected.Bytes()), nil
}
```

IssueNonce + VerifyAndPark 主流程拼图

Redis key `web3:nonce:{userID}:{orderID}`（用户 + 订单双维度）TTL 5min；二次签发自动覆盖旧 nonce；消费走 Lua `GET+DEL`（server 端原子，并发只能一个拿到 1）；Pending 占位 `paydown:web3:pending:{orderID}` TTL 30min。*repository/cache/web3.go:21–26 TTL；:60–89 consumeNonceScript*

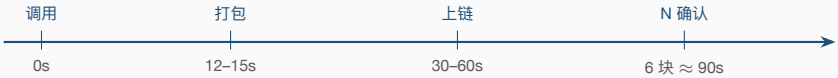
Listing 3: *service/payment_crypto.go:49–148* 节选

```go
// IssueNonce: 校验订单归属 + 状态 + 颁发
order, _ := dao.NewOrderDao(ctx).GetOrderById(req.OrderId, u.Id)
if order.Type != consts.UnPaid { return errors.New("订单状态非未支付") }
nonce, _ := randomNonce() // 16 字节 -> 32 hex
cache.PutWeb3Nonce(ctx, u.Id, req.OrderId, nonce) // SET + 5min TTL

// VerifyAndPark: 消费 nonce -> 验签 -> 事务内写 outbox
if err := cache.ConsumeWeb3Nonce(ctx, u.Id, req.OrderID, req.Nonce); err != nil {
    return err // -1 missing / -2 mismatch -> W3-NONCE-EXPIRED
}
msg := []byte(BuildSignMessage(req.OrderID, req.Nonce, req.ChainID))
ok, _ := web3sig.VerifyPersonalSign(req.WalletAddr, msg, sigBytes)
if !ok { return errors.New("签名与钱包地址不匹配") } // W3-SIG-INVALID
```

链上对账：链是异步的

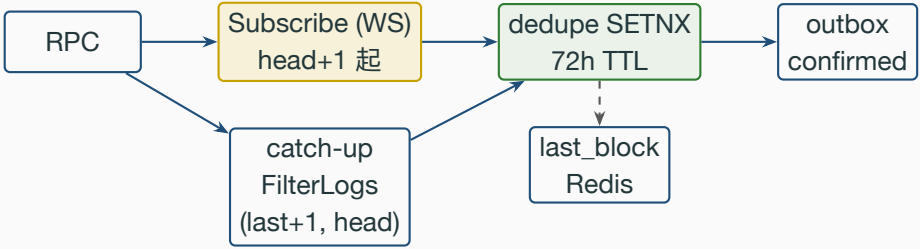
链是异步的：4 个时间锚点



- **0s 调用**：钱包广播 tx 到内存池。
- **12–15s 打包**：validator 把 tx 装进区块。
- **30–60s 1 确认**：业务可显示”已上链”。
- **90s 6 确认**：抗 reorg，金额越大 N 越大（大额 12–20 块）。

链下到链上不是同步调用；用户体验走 ticket 模式：返回 `W3-PENDING` + 客服话术”等待区块确认（约 30–60s）”。*service/web3/listener.go:177–230 connectAndServe head* 推进

listener 的 3 项基本功：重连 / catch-up / 幂等



- 启动 / 重连后先 `catchUp(last+1, head)` 把停机漏掉的事件批量回放。
- 每条 log 落 outbox 前过 `web3:event:{txhash}:{logIdx}` SETNX 72h 幂等。
- 处理一条就 `saveLastBlock`，重启不重做已成功事件。
- 重连退避：**1s** → **2s** → **4s** → **…**→ **60s 封顶**，连上后 reset。

service/web3/listener.go:232–248 catchUp；:296–301 dedupeKey；:316–326 tryClaim

service/web3/listener.go:25–31 eventDedupeTTL=72h；:364–377 backoff

StartPaymentListener 主循环 + connectAndServe

Listing 4: *service/web3/listener.go:177–230 connectAndServe*

```
177 client, _ := l.dial(ctx, l.rpcURL)
178 defer client.Close()
179 head, _ := client.BlockNumber(ctx)
180 last := l.loadLastBlock(ctx)
181 if last == 0 || last > head { last = head }
182 if last < head {
183    l.catchUp(ctx, client, last+1, head) // 回放停机期间漏掉的事件
184 }
185 l.saveLastBlock(ctx, head)
186 logsCh := make(chan types.Log, subscribeBuffer) // = 64
187 sub, _ := client.SubscribeFilterLogs(ctx,
188    l.query(big.NewInt(int64(head+1)), nil), logsCh)
189 defer sub.Unsubscribe()
190 l.backoff.reset() // 连上立即清退避
191 for {
192    select {
193    case <-ctx.Done(): return ctx.Err()
194    case err := <-sub.Err(): return err // -> 外层重连
```

容错矩阵：reorg / 重启 / 重复 / RPC 断

异常	listener 行为	业务影响
RPC 断连	backoff 1s → 60s 指数重连	短暂延迟，不掉单
进程重启	loadLastBlock 后 catchUp(last+1, head)	不掉单
log.Removed	reorg 撤销，跳过	依赖 SETNX 兜底
重复事件	SETNX 72h key 已存在 return	不重复推进订单
RPC 慢	Subscribe 不阻塞主流程，buffer=64	log 缓冲，不丢

复用

deck 04 outbox 总线：listener 写入 `web3.payment.confirmed`，与传统支付的 `order.paid` 走同一条总线，对账可复用。*service/web3/listener.go:155–175 run()* 退避重连；*:218–221 reorg* 跳过 *service/web3/listener.go:296–301 dedupeKey 72h*；*:303–311 outbox.Insert*

链下 vs 链上时间线对比 + ticket UX

Web3 不可能”秒级到账”。业务侧 UX 走 ticket 模式：钱包签名后立即返回 `W3-PENDING` + ticket#，前端轮询订单状态；1 确认前不发货，6 确认（大额 12+）才走 release；客服话术 `W3-PENDING` = ”等待区块确认（约 30–60s）”，不是失败。

Gas 与费用模型：每笔订单实际花多少链费

Gas 概念：合约执行的” 计价器”

- **gas** 是 EVM 操作的计量单位（SLOAD 2100、SSTORE 20000、转账约 21000）。
- 一笔 tx 的链费 $=$ gasUsed $\times$ gasPrice，以 wei 计；mainnet 普通转账 21000 gas，按 30 gwei 算 ≈ 0.00063 ETH。
- EIP-1559 起把 `gasPrice` 拆成 **baseFee**（协议销毁，按区块占用率自适应）+ **priorityFee**（小费给打包者）。
- gomall 链费走**用户自付**：钱包弹窗里用户清楚看到 gas estimate，避免后端补贴被刷。

contracts/Escrow.sol:32–35 custom error 替代 require 字符串（gas 友好） contracts/README.md:91–92 release/refund 用 call 不用 transfer

费用估算：fund vs release vs refund

操作	估算 gas	30 gwei 链费 (ETH)	1 gwei (L2) 链费 (ETH)
fundWithOrderID	≈ 55,000	0.00165	0.000055
release	≈ 35,000	0.00105	0.000035
refund	≈ 35,000	0.00105	0.000035
dispute	≈ 25,000	0.00075	0.000025
ERC-20 approve	≈ 46,000	0.00138	0.000046
ERC-20 transferFrom	≈ 65,000	0.00195	0.000065

mainnet 拥堵时 gasPrice 可上探 100–300 gwei，一笔 fund 链费可达 $10–$30；同样链路放到 Arbitrum / Optimism，因为打包成本走 L1 calldata 摊销，每笔 $0.05–$0.30。

contracts/Escrow.sol:76–82 fundWithOrderID 字节码路径

对比：require 字符串 vs custom error 的 gas 差异

Listing 5: *contracts/Escrow.sol:32–35 custom error；:69 revert WrongAmount*

```
// 旧风格（gas 高）：失败时 ABI-encode 一整段字符串
function fund() external payable {
   require(msg.value == amount, "wrong amount: must equal escrow value");
   // ...
}

// gomall 当前风格（gas 友好）：4 字节 error selector + 紧凑参数
error WrongAmount(uint256 expected, uint256 actual);
function fundWithOrderID(bytes32 _orderID)
   external payable onlyBuyer inState(State.Created)
{
   if (msg.value != amount) revert WrongAmount(amount, msg.value);
   // ... 后续状态推进
}
```

逐行：旧风格 `require` 在 revert 时要把"wrong amount: ..." 整个 UTF-8 串编码进 returndata，路径上每字节 4–16 gas；custom error 只回

Layer 2：把链费降下来，把 TPS 抬上去

为什么 gomall MVP 默认走 L2

- mainnet 单笔 fund 链费 $5–$30，无法支撑 $10 以下小额订单。
- L2 把 calldata 压缩后批量提交回 L1，单笔成本被摊销到 $0.05–$0.30。
- L2 出块更快（Arbitrum 0.25s 软确认 / Optimism 2s / zkEVM 2–5s），用户感知更接近传统支付。
- 安全模型继承 L1（rollup 把 state root 周期性提交回主网），仍是” 区块链安全”，不是” 侧链信任”。

service/payment_crypto.go:39–46 chainID 入签名消息，天然支持多链路由

L1 vs Optimism vs Arbitrum vs Polygon zkEVM

维度	Mainnet	Optimism	Arbitrum	Polygon zkEVM	
类型	L1	Optimistic	Optimistic	ZK rollup	
单笔成本	$5–$30	$0.10–$0.40	$0.05–$0.30	$0.05–$0.20	
软确认	12s	2s	0.25s	2–5s	业务侧最
最终性	6–12 块	7 天挑战期	7 天挑战期	≈ 30 分钟 ZK 证明	
EVM 等价	100%	高	高	高（zkEVM Type 2）	
退出 L1	即时	7 天	7 天	数十分钟	

看重的”软确认 + 链费”两项 L2 全胜；唯一差点的是”提现回 L1”周期长，但 gomall 资金是 **escrow 在 L2 本链内闭环**（买家 L2 入、卖家 L2 出），不踩这个坑。

contracts/README.md:96–100 部署参数中 `amountWei` 与链原生币挂钩

多签仲裁：从单 EOA 升到 2-of-3 多签

单签 arbiter 的业务风险

- 当前 MVP：`arbiter` 是平台一个 EOA，私钥放运营手里。
- **业务风险**：私钥丢失 = 所有 Disputed 订单卡死；私钥被盗 = 攻击者可以无理由把所有托管金 `refund` 到自己控制的地址（因为 buyer 地址是攻击者）一等等，实际上 refund 只会进 buyer，但仲裁人可以**偏向裁决**，相当于平台跑路风险。
- **对外承诺**：$10,000+ 的订单用户不会接受” 平台单签拍板”；监管也不接受。
- 升级：把 arbiter 换成 **Gnosis Safe 2-of-3 多签**，3 个签名人持有，任意 2 个签名才能 release / refund。

contracts/Escrow.sol:10 arbiter immutable；:85–100 release / refund 入口 contracts/README.md:101–102 arbiter 推荐用多签

Safe 调 Escrow.release 的 calldata 模板

Listing 6: *contracts/Escrow.sol:85–91 release()*；通过 *Safe* 转发

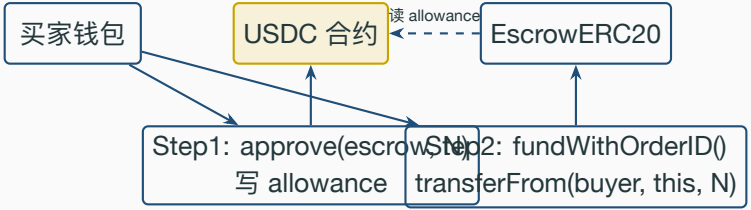
```
// Step 1: 在 Safe 后台或 SDK 里提案一笔交易
// to: address(escrow) // Escrow 合约地址
// value: 0 // release 不带 ETH
// data: abi.encodeWithSignature("release()") // 4 字节 selector
// operation: Call (0) // 普通 call，非 DelegateCall

// Step 2: 收集 2 个签名（owner 私钥 EIP-712 签 SafeTx hash）
// Step 3: 任一签名人调 Safe.execTransaction(...)
// Safe 内部做：
// 1) 校验 threshold 个签名 (2 of 3)
// 2) 检查 nonce 单调递增 (防重放)
// 3) call escrow.release()，此时 msg.sender = Safe
// Step 4: Escrow.release() 校验 msg.sender == arbiter (= Safe addr) 通过
```

逐行：Safe 本质是个**通用 multisig 合约钱包**，对外暴露 `execTransaction`，要求 N-of-M 个 owner EIP-712 签名后才转发 calldata；从 Escrow 视角看 `msg.sender` 就是 Safe 地址（一个合约地址），跟 EOA 没区别；nonce 单调递增防的是**同一组签名换块重放**（提案 N 用

稳定币 ERC-20 直付：approve + transferFrom

ERC-20 支付的两段式：approve + transferFrom



ERC-20 token 是**合约里的账本**，不是链原生币。买家不能"msg.value = N" 直接打 USDC 给合约；必须：(1) 先调 USDC 合约的 `approve(escrow, N)` 授权—在 USDC 内部记 `allowance[buyer][escrow] = N`；(2) 再调 Escrow 的 `fundWithOrderID`，里面用 `IERC20.transferFrom(buyer, this, N)` 拉款。两步 tx，用户得签两次。*contracts/Escrow.sol:76–82 当前 fundWithOrderID 收原生币；ERC-20 版本去掉 payable，加 transferFrom*

EscrowERC20.fundWithOrderID 关键路径

Listing 7: *contracts/EscrowERC20.sol*（规划）—基于 *Escrow.sol:76–82* 改造

```
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract EscrowERC20 {
   IERC20 public immutable token; // USDC / USDT / DAI 合约地址
   uint256 public immutable amount; // USDC 的 smallest unit (6 位小数)
   // buyer / seller / arbiter / state / orderID 同 Escrow.sol

   function fundWithOrderID(bytes32 _orderID)
      external onlyBuyer inState(State.Created)
   {
      orderID = _orderID;
      state = State.Funded;
      // 关键: transferFrom 必须先 approve 过; 返回 false 也算失败
      bool ok = token.transferFrom(msg.sender, address(this), amount);
      require(ok, "transferFrom failed");
      emit Funded(amount);
      emit PaymentConfirmed(_orderID, msg.sender, amount);
   }
}
```

稳定币集成的业务收益与坑点

收益	业务依据
价格稳定	卖家收 USDC 直接进财报，免汇率敞口
对账简单	链上 amount = 法币金额 $\times 10^6$，无需汇率换算
监管接近传统	USDC 发行方 Circle 受美国监管，黑名单地址会被冻结
坑点	**缓解**
两步 tx 用户体验差	EIP-2612 `permit`: 用一次签名授权 + 同 tx transferFrom
USDT 不返回 bool	强制走 SafeERC20.safeTransferFrom
小数位不统一	USDC/USDT 6 位、DAI 18 位、WBTC 8 位，业务层需要 token me
USDC 黑名单	制裁地址会被 Circle 冻结，escrow 内的 USDC 也会被锁

pkg/web3/signature/verify.go:31–67 验签流程对 *token* 类型完全无感

reorg 是什么 + 链是有” 概率最终性” 的

- PoS 以太坊：每 12s 出一个块；**每个 epoch（32 块 ≈ 6.4 分钟）finalize 一次**，finalize 后理论上不再回滚（除非 1/3 验证人作恶 = 损失数十亿美元的 stake）。
- finalize 之前的块都是” 软共识”，遇到分叉时短链会被丢弃 = reorg。
- 业务影响：如果在 1 块确认时就 `release`，下一秒被 reorg 撤销 = 卖家收到的钱” 凭空消失”。
- 对策：按**订单金额**动态选 N 确认。

service/web3/listener.go:218–221 lg.Removed 处理 reorg 撤销的 log service/web3/listener.go:296–301 web3:event:{txhash}:{logIdx} SETNX 72h 兜底

跨链支付路线：CCIP / LayerZero / Wormhole 简评

方案	安全模型	落地难度	适合
CCIP	Chainlink DON 多签	中	大额 / 注重审计
LayerZero	Oracle + Relayer 解耦	低	多链快速接入
Wormhole	Guardian 19-of-26	中	Solana / 非 EVM 链

gomall 短期不

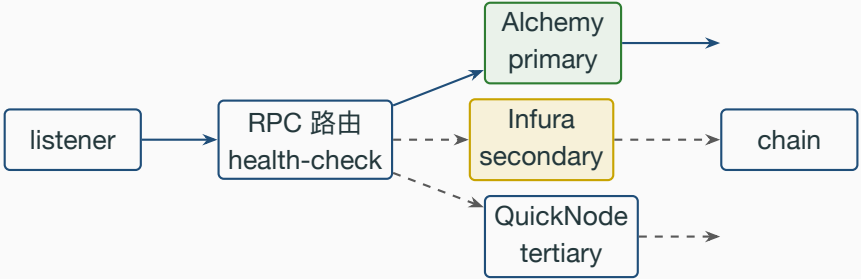
接跨链桥，逻辑是**每条链跑独立 escrow**，避免”跨链桥被盗”风险（行业过去 3 年损失 $25B+ 都来自跨链桥）。长期愿景：买家用任意链的稳定币付款，由桥 / CCIP 在 fulfillment 时换成商家所在链的 USDC，对买家透明。*contracts/README.md:96–100* 每条链独立部署 *Escrow*，*amount* 单位按本链原生 / *token*

KYC / AML：链上身份的合规边界

- **钱包地址 ≠ 实名**：链上地址默认匿名，监管要求 $3,000+ 交易 KYC（FATF Travel Rule）。
- 落地策略：**阶梯式 KYC** —$100 以下免；$100–$3,000 邮箱 + 电话；$3,000+ 强制身份证 + 人脸。
- **OFAC / 制裁名单**：Tornado Cash 等地址必须拒绝；接 Chainalysis / TRM Labs API，验签前过滤。
- **链上分析**：可疑路径（Tornado / 黑客地址 → 买家地址 → 商家）触发风控冻结。
- **多签 + KYC 共生**：仲裁人多签里至少 1 个签名人要持有” 合规岗位” 权限，链上的每一次 release 都有合规可追溯证据。

service/payment_crypto.go:116–124 VerifyPersonalSign 是合规过滤的天然插入点

RPC 节点容灾：多源 + 自动 failover



- 单 RPC 厂商挂掉 listener 就静默 = 链上 confirm 收不到 = 订单卡在 pending。
- 策略：环境变量 `WEB3_RPC_URL` 改成逗号分隔列表，listener 按 health-check 顺序切换。
- 失败计数：连续 3 次 dial error 或 5 次 subscribe drop 切下一家。
- 切换不重置 `last_block`，catch-up 自动补齐换源期间漏掉的事件。

service/web3/listener.go:113–117 envRPCURL 单源；多源升级走 strings.Split + 轮询

业务边界：业务码 / 角色 / 合规

Web3 业务码 + 客服话术

业务码	含义	客服话术
W3-NONCE-EXPIRED	nonce 5min 过期或被消费	” 请重新发起支付”
W3-SIG-INVALID	签名地址与提交不匹配	” 请用绑定钱包重试”
W3-PENDING	链上未确认	” 等待区块确认（约 30–60s）”
W3-REORG	链上 reorg 撤销	” 网络回滚，已自动退款”

业务码不暴露技术细节，但要给客服**足够区分**的话术。

`W3-PENDING` 与 `W3-NONCE-EXPIRED` 表面像，应对策略完全不同：前者等链确认，后者重发 nonce。*repository/cache/web3.go:29–33 ErrWeb3NonceMissing / ErrWeb3NonceMismatch*

各角色关心什么

角色	关心	看哪里
用户	钱安全吗	合约审计报告 + 多签 + 退款流程
商家	何时到账	链上确认数 + release 时机
客服	链上失败咋办	W3-* 业务码对应处理 SOP
SRE	节点稳定否	RPC 多源切换 + listener 健康
法务	合规边界	KYC + 多签 + 司法管辖
财务	链上链下并存对账	outbox + 链上事件 + 法币入账

`outbox` 总线让财务对账

拿到**统一格式**的事件：channel = fiat / web3-eth / web3-usdc，按 `order_id` 维度 join。**业务边界**：链 / 币 / gas 三选一组合，MVP 推荐 **L2 + 稳定币 + 用户自付 gas**；正式运营后逐步上 gas 补贴 + 多链路由。*contracts/README.md:88–103* 多签建议；*service/payment_crypto.go:41* 多 *chainID* 接口

风险与合规 + Arbiter 多签升级路线图

- **合约 immutable**：bug 只能换合约；缓解：第三方审计 + 充分单测。
- **私钥丢失**：链上无” 重置密码”；缓解：仲裁人核实身份后 refund 到新地址。
- **gas 飙升**：mainnet 拥堵 tx 失败；缓解：默认 L2，fallback mainnet。
- **Reorg**：浅确认下 release 可能撤销；缓解：大额提高 N 确认 + outbox 兜底。
- **AML / OFAC**：制裁名单地址必须拒绝；缓解：验签阶段地址过滤。

阶段	arbiter 形态	适用金额上限
MVP	平台 EOA	等值 $100 以内
2	2-of-3 多签	等值 $10,000 以内（Gnosis Safe）
3	DAO 治理	超大额 / 跨组织

service/web3/listener.go:218–221 reorg 跳过；contracts/README.md:88–103 arbiter 多签

从技术决策回看业务依据 + 最终目标

技术选择	业务依据
EIP-191 personal_sign	钱包弹窗让用户看到明文，钓鱼可识别
nonce 走 Redis + Lua	防重放门槛要原子，单实例语义不够
chainID 入签名消息	防 mainnet / L2 跨链重放
合约 emit PaymentConfirmed	listener 订一个 event 就能精确对账
listener 写 outbox 而非直推订单	复用 deck 04 的最终一致体系
SETNX 72h 事件级幂等	reorg / 重连 / 重启不会重复推进
arbiter 走多签路线	单签平台跑路风险无法对外解释

业务价值不在”潮”，而在于**把”第三方”从”中介机构”换成”可审计的代码”**。

routes/router.go:123–127 Web3 路由接入位

面试 Q&A（上）

Q1：链 reorg 把已 release 的资金撤销了怎么办?

A：金额越大 N 确认越多（mainnet 大额走 32 块 = 1 epoch finalize；L2 大额等回 L1 finalize)；listener 监听 `lg.Removed` 跳过 reorg log，订单状态不会被错误推进；已发货走对账兜底 + 仲裁人介入。

service/web3/listener.go:218–221

Q2：用户钱包私钥丢了订单怎么处理?

A：链上没有”重置密码”。资金在 Funded 态可由仲裁人核实身份后 refund 到新地址；已 Released 给商家则只能走链下补偿。

contracts/Escrow.sol:94–100 refund 入口可由 arbiter 触发

Q3：gas 飙升导致用户支付 tx 失败怎么补救?

A：默认路由到 L2；nonce 5min 内允许用户重新提交带更高 gas 的 tx；超时则重新颁发 nonce + 签名。EIP-1559 钱包会自动调高 maxFee 减少这种失败。

repository/cache/web3.go:21–26 Web3NonceTTL=5min

Q4：custom error 和 require 字符串对 gas 影响有多大?

面试 Q&A（中）

Q5：和传统第三方支付的对账如何并存？

A：两条通道都把支付事件落到同一条 outbox 总线，事件加 channel 字段（fiat / web3-eth / web3-usdc）。对账任务按 `order_id` join 法币入账 + 链上 confirmed event。

service/payment_crypto.go:129–144 outbox 写入逻辑

Q6：为什么 nonce 必须后端发，前端自己生成不行吗？

A：前端可生成的 nonce 后端没法保证唯一性也没法做时效控制；攻击者拿到一次有效签名后可无限重放。后端 Redis + Lua 原子 GET+DEL 才能保证” 一次性”。

repository/cache/web3.go:60–89 consumeNonceScript

Q7：listener 进程重启会不会丢事件？

A：不会。`loadLastBlock` 从 Redis 拿到最后处理过的块号，`catchUp(last+1, head)` 用 `FilterLogs` 批量回放停机期漏掉的事件，再叠加 SETNX 72h 幂等，重启不会重做也不会丢做。

service/web3/listener.go:188–197 catchUp；:316–326 tryClaim

Q8：单签 arbiter 上线后多久应该切多签？

面试 Q&A（下）

Q9：稳定币集成的最大业务卡点是什么？

A：两步 tx (approve + transferFrom) 用户体验差。落地路径：(1) 短期接受两步 tx，前端引导用户一次授权较大额度；(2) 中期接 EIP-2612 `permit`，把 approve 改成一次签名同 tx 内做 transferFrom；(3) 长期接 ERC-4337 账户抽象，把多笔操作合成一笔。

contracts/EscrowERC20.sol（规划）—基于 *Escrow.sol:76–82* 改造

Q10：Optimistic rollup 的 7 天挑战期对 gomall 有什么实际影响？

A：对 L2 内闭环交易**无影响**（买家 L2 入、卖家 L2 出，资金一直在 L2 上）；只有当卖家想把 L2 的稳定币提回 L1 时才碰到 7 天。变通：走第三方 fast-bridge（Hop / Across），用流动性提供方贴前 7 天的钱给卖家，平台 / LP 收 0.1%–0.3% 手续费。

contracts/README.md:96–100 escrow 在本链闭环

Q11：RPC 节点中途切换会不会导致漏事件？

A：不会。`last_block` 持久化在 Redis，换源后 `loadLastBlock` + `catchUp` 自动补齐换源期间的所有 log；再叠加 SETNX 72h 幂等，重复推送也只生效一次。

service/web3/listener.go:328–352 loadLastBlock / saveLastBlock

代码位置一览（上）

Escrow 合约 + 状态机 *contracts/Escrow.sol:7–113*

immutable 参数 / custom error *contracts/Escrow.sol:8–11；:32–35*

fundWithOrderID 入金 *contracts/Escrow.sol:76–82*

release / refund / dispute *contracts/Escrow.sol:85–107*

钱包验签 EIP-191 *pkg/web3/signature/verify.go:31–82*

Nonce 颁发 + 验签业务 *service/payment_crypto.go:49–160*

签名消息模板 / chainID 防重放 *service/payment_crypto.go:39–46*

Web3 Redis 协议（nonce/pending） *repository/cache/web3.go:21–101*

Lua 原子 GET+DEL *repository/cache/web3.go:60–89*

代码位置一览（下）

链上 listener 主循环 *service/web3/listener.go:108–230*

catch-up 补块 *service/web3/listener.go:232–248*

reorg 跳过 *service/web3/listener.go:218–221*

事件级幂等 SETNX 72h *service/web3/listener.go:296–326*

last_block 持久化 *service/web3/listener.go:328–352*

退避重连 *service/web3/listener.go:360–377 backoff 1s→60s*

Outbox 总线写入 *service/payment_crypto.go:129–144*

API 路由接入位 *routes/router.go:123–136*

crypto handler *api/v1/paydown_crypto.go:14–52*

合约部署 / 多签建议 *contracts/README.md:1–104*

现状：合约 + binding 在 feat/web3-escrow PR；**未上链**，上线前需第三方审计 + 切多签。